

自适应归一化 AdaLN 讲义

从 AdaIN / FiLM 到 DiT 的 adaLN-Zero：一种”神奇”的条件注入结构

Bowen 的知识库 · Research Notes

2026 年 6 月 20 日

摘要

AdaLN (Adaptive Layer Normalization, 自适应层归一化) 是当今扩散 Transformer (DiT)、Stable Diffusion 3 (MMDiT) 等主流生成模型注入条件 (时间步、类别、文本、动作等) 的默认机制。它”神奇”的地方在于：用一个几乎零开销的逐通道仿射调制，就能在保持归一化稳定性的同时注入条件，并且其 **adaLN-Zero** 变体通过零初始化把每个 Transformer 块初始化为恒等映射，使大模型训练异常稳定、FID 显著优于交叉注意力。本讲义结合 AdaIN [1]、FiLM [3]、DiT [4] 等原始论文，推导其数学形式、梳理谱系演化、剖析官方代码，并给出设计直觉与适用边界。

目录

1 动机：归一化层里藏着”条件”	1
1.1 归一化回顾	1
2 谱系演化：四步走到 AdaLN (结合原文)	2
2.1 第一步：Conditional Instance Norm (CIN, Dumoulin 2017)	2
2.2 第二步：AdaIN (Huang & Belongie, ICCV 2017)	2
2.3 第三步：FiLM (Perez et al., AAAI 2018)	2
2.4 第四步：AdaLN (DiT, Peebles & Xie, ICCV 2023)	3
3 核心：adaLN-Zero 为什么”神奇”	3
3.1 思想来源：把残差块初始化为恒等映射	3
3.2 adaLN-Zero 的构造	3
3.3 完整的 DiT 块公式	4
3.4 为什么有效：原文的实证	4
4 代码剖析：DiT 官方实现	5
5 设计直觉、适用边界与对比	6
5.1 为什么”调制归一化”比”直接拼接/相加”更优雅	6
5.2 局限：AdaLN 是”全局/逐通道”的，不擅长空间对齐	7
5.3 能否把 AdaLN 扩成逐位置 $\mathbb{R}^{B \times N \times C}$? SPADE 与 adaLN-single 的双向证据	7

1 动机：归一化层里藏着”条件”	2
5.4 与交叉注意力的取舍	8
6 小结	8

1 动机：归一化层里藏着”条件”

我们先建立一个核心直觉，它是理解整条技术线的钥匙：

洞察 1.1. 归一化层 (BN/IN/LN) 做两件事：(1) 用统计量 (μ, σ) 把激活标准化；(2) 用仿射参数 (γ, β) 把它重新缩放平移。大量研究发现：标准化去除的是与”内容/实例”无关的整体分布信息，而仿射参数 (γ, β) 恰恰承载了”风格 / 域 / 条件”。

这意味着：如果我们想让网络”按条件 **c** 生成”，一个极其经济的做法不是改输入、加注意力，而是让条件 **c** 去控制归一化层的仿射参数 (γ, β) 。这就是从 AdaIN 到 AdaLN 一脉相承的中心思想。

1.1 归一化回顾

对一个特征张量，记其在归一化维度上的均值方差为 μ, σ 。标准（带仿射的）归一化为

$$\text{Norm}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu}{\sigma} + \beta, \quad (1)$$

其中 γ, β 是可学习且固定的参数（训练完就不再变）。不同归一化的区别只在于 μ, σ 在哪些维度上统计：

- **BatchNorm**：跨 batch + 空间统计（每通道一个）。
- **InstanceNorm (IN)**：每个样本、每个通道、跨空间统计。
- **LayerNorm (LN)**：每个样本（每个 token）跨特征维统计——Transformer 的标配。

接下来的所有方法，本质都是把固定的 (γ, β) 换成”由条件动态生成的” (γ, β) 。

2 谱系演化：四步走到 AdaLN（结合原文）

2.1 第一步：Conditional Instance Norm (CIN, Dumoulin 2017)

为了让一个网络支持多种风格，Dumoulin 等人提出 CIN：为每个风格 s 学一组独立的 γ^s, β^s 。AdaIN 原文 [1] 引述其公式（式 7）为

$$\text{CIN}(\mathbf{x}; s) = \gamma^s \left(\frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} \right) + \beta^s. \quad (2)$$

局限非常明显：风格数 S 固定（原文 $S = 32$ ），无法泛化到任意新风格——因为 (γ^s, β^s) 是查表得到的离散参数。

2.2 第二步：AdaIN (Huang & Belongie, ICCV 2017)

AdaIN [1] 的灵光一闪：既然 IN 的统计量本身就编码风格，那干脆用风格图像 y 的统计量 $(\mu(y), \sigma(y))$ 直接当作仿射参数，连学习都不需要：

$$\text{AdaIN}(\mathbf{x}, y) = \sigma(y) \left(\frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} \right) + \mu(y). \quad (3)$$

其中 $\mathbf{x}$ 是内容特征、 y 是风格特征，统计量跨空间位置计算。原文强调：**AdaIN 没有任何可学习仿射参数**，它把”换风格”变成了”把内容的通道统计对齐到风格的通道统计”。这是”条件 $\rightarrow$ 仿射参数”思想第一次以自适应 (adaptive, 参数随输入而变) 的形式出现。StyleGAN 即用 AdaIN 注入风格向量。

洞察 2.1. AdaIN 的贡献是把”条件”与”仿射参数”划等号并即时计算。但它的条件被限定为”另一张图的统计量”。下一步要做的，是把条件一般化为任意信号，并把 (γ, β) 改为由一个网络回归出来。

2.3 第三步：FiLM (Perez et al., AAAI 2018)

FiLM [3] 把上面的思想推到最一般的形式：用一个**条件生成网络**从任意条件 $\mathbf{x}_i$ （文本、类别、问题语句……）回归出逐特征的 $\gamma_{i,c}, \beta_{i,c}$,

$$\gamma_{i,c} = f_c(\mathbf{x}_i), \quad \beta_{i,c} = h_c(\mathbf{x}_i), \quad (4)$$

然后对目标网络的中间特征 $F_{i,c}$ 做逐特征仿射调制 (feature-wise affine)：

$$\text{FiLM}(F_{i,c} | \gamma_{i,c}, \beta_{i,c}) = \gamma_{i,c} F_{i,c} + \beta_{i,c}. \quad (5)$$

原文明确指出：**FiLM 是 Conditional Normalization 的一般化**——它不再绑定到归一化的统计量， (γ, β) 可以是任意条件的函数。FiLM 能够放大、缩小、取反、关闭、选择性阈值化特征，从而以极小的代价控制一个深层网络的计算。原文还观察到一个重要现象：**FiLM 自身是线性的**，其表达力来自它后面的非线性层——单个 FiLM 层放在 CNN 早期，就能取得接近”4 个 FiLM 层遍布全网”的效果。

2.4 第四步：AdaLN (DiT, Peebles & Xie, ICCV 2023)

DiT [4] 把 FiLM 思路搬进 Transformer：既然 Transformer 用 LayerNorm，那就让条件 $\mathbf{c}$ （时间步嵌入 + 类别/文本嵌入）去回归 LN 的仿射参数。这就是 **AdaLN**。具体地，先把 LayerNorm 设为无自带仿射 (`elementwise_affine=False`)，再用一个小 MLP 从 $\mathbf{c}$ 回归出 (γ, β) ：

$$(\gamma, \beta) = \text{MLP}(\mathbf{c}), \quad \text{AdaLN}(\mathbf{x}, \mathbf{c}) = \gamma(\mathbf{c}) \odot \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} + \beta(\mathbf{c}). \quad (6)$$

与交叉注意力相比，AdaLN 几乎不增加 Gflops (DiT 原文：adaLN 118.6 vs cross-attn 137.6 Gflops)，却把条件高效地广播到每个 token 的每个通道。

3 核心：adaLN-Zero 为什么”神奇”

AdaLN 已经好用，但 DiT 真正的杀手锏是 **adaLN-Zero**。它的”神奇”来自一个看似微不足道、实则深刻的初始化技巧。

方法	(γ, β) 来源	条件类型	关键特征 / 局限
LN/IN(仿射)	可学习固定	无条件	训练后不变
CIN	查表 γ^s, β^s	离散风格 s	风格数固定，不能泛化
AdaIN	风格图统计 $\mu(y), \sigma(y)$	任意风格图	无可学习参数；条件限于”统计量”
FiLM	条件网络回归 $f(\mathbf{c}), h(\mathbf{c})$	任意条件	CN 的一般化；线性，靠后续非线性赋能
AdaLN	MLP($\mathbf{c}$) 回归	时间步/类别/文本	用于 Transformer LN；近零算力
adaLN-Zero	MLP($\mathbf{c}$) + 门控 α	同上	零初始化 $\rightarrow$ 块 = 恒等；最稳最优

表 1: 自适应归一化谱系：本质都是”把固定 (γ, β) 换成由条件生成的 (γ, β) ”。

3.1 思想来源：把残差块初始化为恒等映射

DiT 原文 [4] 引述了两条经验：

- Goyal 等人发现，在监督学习里把每个残差块最后一个 BN 的 γ 零初始化，能加速大规模训练；
- 扩散 U-Net 普遍把每个块残差连接前的最后一个卷积零初始化。

共同的原理是：让残差分支在训练开始时输出 0，于是 $\mathbf{x} \leftarrow \mathbf{x} + 0 = \mathbf{x}$ ，整个块成为**恒等函数 (identity)**，给优化一个极其平稳的起点。

3.2 adaLN-Zero 的构造

在 AdaLN 回归 (γ, β) 的基础上，adaLN-Zero 额外回归一个逐通道门控 α ，作用在残差相加之前：

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha(\mathbf{c}) \odot \text{SubLayer}(\text{AdaLN}(\mathbf{x}, \mathbf{c})), \quad (7)$$

其中 $\text{SubLayer} \in \{\text{Self-Attn}, \text{MLP}\}$ 。关键一招：把每个 *block* 内部那个回归 (γ, β, α) 的调制 MLP 的末层 *Linear* (MLP_{adaLN} 的最后一层) 权重和偏置全部零初始化。使初始时 $\alpha = \mathbf{0}$ (同时 γ 对应的有效缩放为 1、 $\beta = \mathbf{0}$)，于是

$$\mathbf{x} \leftarrow \mathbf{x} + \mathbf{0} \odot (\cdots) = \mathbf{x},$$

该 block 在训练第 0 步即为恒等映射。

注 3.1 (这里的”最后一层”不是”最后一个 block”)。要分清两个”最后”：被清零的是每个 *block* 内部调制 MLP ([SiLU, Linear]) 的末层 *Linear*，而不是整个网络的最后一个 DiT block。代码里 for block in self.blocks 对所有 block 都执行了这一清零，因此所有 block 在初始时都是恒等映射，并非只有最后一块。此外 DiT 还把最终输出层 (FinalLayer) 的调制 MLP 末层与最后的投影 linear 一起清零，使整个网络在第 0 步预测的噪声恒为 0——一个极其平稳的训练起点。

注 3.2 (全恒等为何不妨碍学习)。”所有 block 恒等”只是初始化，不影响表达能力，原因在于梯度流向。对 $\mathbf{x} \leftarrow \mathbf{x} + \alpha \odot S$ ($S = \text{SubLayer}(\cdots)$)，注意/MLP 子层为正常非零初始化，故 $S \neq \mathbf{0}$)：第

0 步 $\frac{\partial L}{\partial \alpha} = \frac{\partial L}{\partial \mathbf{x}_{\text{out}}} \odot S \neq \mathbf{0}$, 门控 α 立刻被更新而离开 0; 而子层内部权重的梯度 $\frac{\partial L}{\partial \theta_S} \propto \alpha \odot \frac{\partial S}{\partial \theta_S}$ 在 $\alpha = 0$ 时为 0、待 α 变非零后才开始更新。于是网络先打开”阀门” α 、再逐步激活各残差分支, 从恒等平滑”长出”能力 (ResNet zero-init / Fixup / SkipInit 思想)。恒等初始化的收益随深度累积, 故应对所有 block 统一施加, 而非只做一部分。

3.3 完整的 DiT 块公式

一个 adaLN-Zero DiT 块为每个子层各回归一组 (shift, scale, gate), 共 6 个向量:

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha_1 \odot \text{Attn}\left(\text{modulate}(\text{LN}(\mathbf{x}), \beta_1, \gamma_1)\right), \quad (8)$$

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha_2 \odot \text{MLP}\left(\text{modulate}(\text{LN}(\mathbf{x}), \beta_2, \gamma_2)\right), \quad (9)$$

$$\text{其中 } (\beta_1, \gamma_1, \alpha_1, \beta_2, \gamma_2, \alpha_2) = \text{MLP}_{\text{adaLN}}(\mathbf{c}), \quad (10)$$

$$\text{modulate}(\mathbf{x}, \beta, \gamma) \triangleq \mathbf{x} \odot (1 + \gamma) + \beta. \quad (11)$$

注 3.3. 注意式 (11) 用的是 $(1 + \gamma)$ 而非 γ 。这是工程上的关键细节: MLP 零初始化时 $\gamma = \mathbf{0}$, 于是缩放因子为 $1 + 0 = 1$, 即初始恰好是标准 LayerNorm, 不引入任何额外缩放; 条件的作用是在 1 附近做增量调制, 数值更稳定。

3.4 为什么有效: 原文的实证

DiT 原文用最高算力的 DiT-XL/2, 对比四种条件注入块 (in-context / cross-attention / adaLN / adaLN-Zero), 测训练过程的 FID, 结论非常干脆:

- **adaLN-Zero 的 FID 同时低于交叉注意力与 in-context**, 且最省算力;
- 在 400K 迭代时, adaLN-Zero 的 FID 几乎只有 in-context 的一半——说明条件注入方式本身关键地影响模型质量;
- **初始化也很关键**: adaLN-Zero (块初始化为恒等) 显著优于普通 adaLN。

正因如此, DiT 全文后续模型一律采用 adaLN-Zero; 该结构也被 Stable Diffusion 3 的 MMDiT 等继承。

4 代码剖析: DiT 官方实现

下面是 DiT 官方 models.py 的核心片段, 逐行对应上面的公式。

```

1 def modulate(x, shift, scale):
2     # 注意是 (1 + scale): 零初始化时缩放=1, 等价标准 LN
3     return x * (1 + scale.unsqueeze(1)) + shift.unsqueeze(1)
4
5 class DiTBlock(nn.Module):
6     """带 adaLN-Zero 条件的 DiT 块"""
7     def __init__(self, hidden_size, num_heads, mlp_ratio=4.0, **kw):
8         super().__init__()
9         # LN 关掉自带仿射, 仿射交给条件回归
10        self.norm1 = nn.LayerNorm(hidden_size, elementwise_affine=False, eps=1e-6)

```

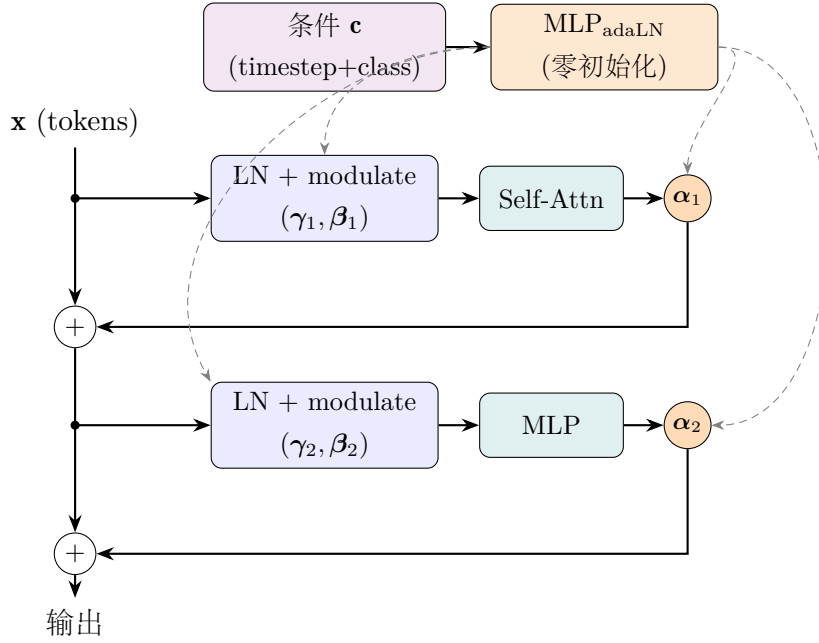


图 1: adaLN-Zero DiT 块: 左侧竖直线为残差主流, 输入 $\mathbf{x}$ 在分叉点 (实心圆) 旁路进入 “LN+modulate $\rightarrow$ 子层 $\rightarrow$ 门控 α ” 后由 $+$ 汇回主流。橙色 $\text{MLP}_{\text{adaLN}}$ 从条件 $\mathbf{c}$ 回归 6 组参数 $(\gamma_i, \beta_i, \alpha_i)$ (虚线) 并零初始化; 初始 $\alpha_i = \mathbf{0}$ 使两个子块输出为 0, 整块成为恒等映射。

```

11     self.attn = Attention(hidden_size, num_heads=num_heads, qkv_bias=True, **kw)
12     self.norm2 = nn.LayerNorm(hidden_size, elementwise_affine=False, eps=1e-6)
13     self.mlp = Mlp(in_features=hidden_size,
14                     hidden_features=int(hidden_size*mlp_ratio),
15                     act_layer=lambda: nn.GELU(approximate="tanh"))
16     # 从条件 c 回归 6*hidden 个调制参数
17     self.adaLN_modulation = nn.Sequential(
18         nn.SiLU(),
19         nn.Linear(hidden_size, 6 * hidden_size, bias=True))
20
21     def forward(self, x, c):
22         # 切成 6 份: 注意力子层与 MLP 子层各 (shift, scale, gate)
23         (shift_msa, scale_msa, gate_msa,
24          shift_mlp, scale_mlp, gate_mlp) = self.adaLN_modulation(c).chunk(6, dim=1)
25         x = x + gate_msa.unsqueeze(1) * self.attn(modulate(self.norm1(x), shift_msa,
26                                                            scale_msa))
27         x = x + gate_mlp.unsqueeze(1) * self.mlp(modulate(self.norm2(x), shift_mlp,
28                                                            scale_mlp))
29         return x
30
31     # 初始化: 对【所有】block, 把其调制 MLP 的【末层 Linear】清零 => 每块=恒等
32     for block in self.blocks:
33         nn.init.constant_(block.adaLN_modulation[-1].weight, 0)
34         nn.init.constant_(block.adaLN_modulation[-1].bias, 0)

```

```

33
34 # 再把【最终输出层】也清零 => 整网第 0 步输出恒为 0
35 nn.init.constant_(self.final_layer.adaLN_modulation[-1].weight, 0)
36 nn.init.constant_(self.final_layer.adaLN_modulation[-1].bias, 0)
37 nn.init.constant_(self.final_layer.linear.weight, 0)
38 nn.init.constant_(self.final_layer.linear.bias, 0)

```

要点对照：

- (1) `elementwise_affine=False`: LN 只做标准化，仿射全部交给条件——呼应洞察 1.1。
- (2) `6 * hidden_size`: 一个块两个子层 $\times$ 每子层 (shift, scale, gate) = 6 组。
- (3) `modulate` 用 $(1 + \text{scale})$: 保证零初始化时等价于普通 LN。
- (4) `gate*` (即 α) 乘在子层输出上、加回残差前: 零初始化它就让块 = 恒等。
- (5) `constant(..., 0)`: 把 adaLN MLP 末层清零，是 adaLN-Zero 的“Zero”所在。

5 设计直觉、适用边界与对比

5.1 为什么”调制归一化”比”直接拼接/相加”更优雅

- **解耦内容与条件**: LN 先把每个 token 的特征标准化（去掉与内容相关的整体尺度），再由条件统一施加”风格”。条件控制的是分布而非具体数值，泛化更好。
- **近零算力、强可扩展**: 只需一个共享 MLP 回归调制参数，参数量相对骨干极小，利于 scaling (DiT 的核心卖点)。
- **训练稳定**: 恒等初始化让深层 Transformer 从一个良态起点出发，避免早期条件噪声破坏表示。

5.2 局限: AdaLN 是”全局/逐通道”的，不擅长空间对齐

先用张量形状把”逐通道”讲清。设特征为 $\mathbf{x} \in \mathbb{R}^{B \times N \times C}$ (N 为 token / patch 数，即空间位置维)，全局条件为 $\mathbf{c} \in \mathbb{R}^{B \times C}$ (每个样本一个向量，无 N 维)。adaLN 的调制 MLP 是 $\text{Linear}(C \rightarrow 6C)$:

$$\mathbf{c} \xrightarrow{\text{MLP}} \mathbb{R}^{B \times 6C} \xrightarrow{\text{chunk}} \underbrace{(\gamma_1, \beta_1, \alpha_1, \gamma_2, \beta_2, \alpha_2)}_{\text{每个} \in \mathbb{R}^{B \times C}}.$$

调制时把它们 `unsqueeze` 成 $\mathbb{R}^{B \times 1 \times C}$ 再沿 N 广播:

$$y_{b,i,c} = \gamma_{b,c} \cdot \hat{x}_{b,i,c} + \beta_{b,c} \quad (\text{右边无位置下标 } i).$$

即同一样本内所有位置 i 共享同一组 (γ, β, α) ，只在通道维 C 上逐元素不同——这就是”逐通道、空间共享”；输出形状与输入相同 ($B \times N \times C$)。

因此 γ, β 只能”整体地拧每个特征通道的旋钮”、对所有 patch 一视同仁:

- 它非常适合时间步、类别、池化后的文本/动作向量这类全局条件；
- 但对逐像素 / 逐位置对齐的稠密条件（深度图、分割、相机 Plücker 图、历史帧）力不从心——无法表达”左上放猫、右下放狗”这类位置相关控制；这类条件应改用通道拼接、交叉注意力或 ControlNet 式零卷积旁路（见配套文档《条件注入机制综述》）。

洞察 5.1. 选型法则：全局/低维条件 $\Rightarrow$ AdaLN(-Zero)；空间/逐位置稠密条件 $\Rightarrow$ 拼接 / 交叉注意力 / 零卷积旁路。现代多模态 DiT (如 MMDDiT) 常混用：文本走联合注意力，时间步走 AdaLN。

5.3 能否把 AdaLN 扩成逐位置 $\mathbb{R}^{B \times N \times C}$? SPADE 与 adaLN-single 的双向证据

既然“空间共享”是短板，一个自然的想法是：让调制 MLP 直接输出逐位置的 $(\gamma, \beta, \alpha) \in \mathbb{R}^{B \times N \times C}$ 做更细粒度的调控。文献其实从两个相反方向都已给出答案。

正向——这正是 SPADE。 SPADE (空间自适应归一化, [6]) 比 DiT 更早，就把 γ, β 升级为与特征图同尺寸的张量、逐位置调制：

$$\text{out}_{n,c,y,x} = \gamma_{n,c,y,x} \hat{h}_{n,c,y,x} + \beta_{n,c,y,x},$$

其实现 `out = norm(x)*(1+gamma)+beta` 与 DiT 的 `modulate` 完全同形，唯一差别是 γ, β 带空间下标。但 SPADE 成立有一个**决定性前提**：它的逐位置 γ, β 是从一张空间条件图（分割图）用卷积算出来的——**信息本身必须是空间的**。

关键阻碍——全局条件 + MLP 变不出逐位置信息。 DiT 的 $\mathbf{c}$ 是全局向量 $\mathbb{R}^{B \times C}$ 、不含 N 维。以全局向量为输入的函数无法凭空产生位置相关的输出：要让输出带 N ，只能再喂入带 N 的东西（通常是位置编码），此时 γ_i, β_i 退化为“位置 i 的固定学习模板、与内容无关”，不再 adaptive，且把模型与固定序列长度/分辨率绑死。要真正实现 **content-adaptive** 的逐位置调制，就必须引入空间条件——而那时你做的已经是 SPADE / 交叉注意力 / ControlNet，而非 adaLN。

反向——PixArt- α 的 adaLN-single 量化了代价。 [7] 发现 adaLN 的线性投影在标准 DiT 中已占 **27% 参数**（这还是最省的 $\mathbb{R}^{B \times C}$ 粒度）。它索性反着做：只在首块算一套全局 $\bar{S} = f(t)$ 、其余块共享 + 各自一个可学习偏置 $E^{(i)}$ (adaLN-single)，结果**参数省 26%、显存省 21% 且质量不降**。这说明对全局条件，趋势是往更少粒度走；若反向膨胀到 $\mathbb{R}^{B \times N \times C}$ ，代价只会相反地放大。

naive 扩展的四条后果。

- (1) **信息：**全局条件下，逐位置变化要么来自内容（须引入空间条件，坍缩为 SPADE/注意力），要么仅来自位置编码（固定模板、非自适应）——单纯改输出维度并不会凭空增加可控信息。
- (2) **参数 / 算力：**调制生成器须逐 *token* 产出 $\mathbb{R}^{B \times N \times 6C}$ ，从“近零开销的广播”变成“对整条序列起作用的网络”，adaLN 的效率卖点尽失，长序列还显著增显存。
- (3) **归纳偏置 / 过拟合：**“沿 N 广播”本身是一种正则；去掉后自由度暴涨、更吃数据，且与 self-attention“处理位置间交互”的职责功能重叠，易冗余甚至不稳（与 [8] 在时空纠缠 DiT 中的观察一致）。
- (4) **分辨率泛化：**全局粒度天然分辨率无关；逐位置量一旦绑定 N 索引，换分辨率/序列长度即对不上。

（训练稳定性本身不受影响——仍可零初始化恒等起步；问题全在以上四项。）

洞察 5.2. 正确路线是”按条件的对齐结构选注入机制”，而非硬把 adaLN 扩成逐位置：全局条件保持逐通道 adaLN（甚至像 PixArt 那样共享精简）；空间/稠密条件交给 SPADE（空间图 $\rightarrow$ 空间 γ, β ）、交叉注意力（逐 token）或 ControlNet（逐像素残差）。

5.4 与交叉注意力的取舍

	AdaLN-Zero	Cross-Attention
算力开销	极低（近零 Gflops）	较高（额外 KV 投影与注意力）
条件粒度	全局/逐通道，空间共享	可逐 token 对齐，支持变长序列
适配条件	时间步、类别、标量/池化向量	文本序列、离散动作、变长结构条件
训练稳定性	恒等初始化，非常稳	取决于初始化（可配零初始化）
DiT 实证	FID 最优、最省算力	FID 次于 adaLN-Zero

6 小结

1. AdaLN 的思想源头是洞察 1.1：归一化的仿射参数承载条件/风格，于是”按条件生成”=”让条件控制 (γ, β) ”。
2. 谱系：CIN（离散查表） $\rightarrow$ AdaIN（用风格统计、无参数、式 (3)） $\rightarrow$ FiLM（任意条件回归、CN 的一般化、式 (5)） $\rightarrow$ AdaLN（搬进 Transformer LN）。
3. adaLN-Zero 的”神奇”= 在 AdaLN 上加逐通道门控 α + 零初始化，把每个块初始化为恒等映射；细节上用 $(1 + \gamma)$ 保证初始等价标准 LN。
4. 实证：adaLN-Zero 比交叉注意力、in-context 都更优且更省算力，已成 DiT / SD3 的默认。
5. 边界：AdaLN 适合全局条件；空间对齐的稠密条件需另选注入算子。
6. 想做逐位置 $\mathbb{R}^{B \times N \times C}$ 调制就是 SPADE（须空间条件），而全局条件下硬扩只会丢失效率、破坏归纳偏置与分辨率泛化；PixArt 的 adaLN-single 反向印证”全局条件宜更省粒度”。

一句话总结：AdaLN 把”条件注入”优雅地化归为”让条件去写归一化的仿射参数”，而 adaLN-Zero 再用”零初始化的恒等起点”换来了大模型训练的稳定与质量——小机制，大杠杆。

参考文献

- [1] X. Huang, S. Belongie. *Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization (AdaIN)*. ICCV 2017. <https://arxiv.org/abs/1703.06868>
- [2] V. Dumoulin, J. Shlens, M. Kudlur. *A Learned Representation For Artistic Style (Conditional Instance Normalization)*. ICLR 2017. <https://arxiv.org/abs/1610.07629>
- [3] E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville. *FiLM: Visual Reasoning with a General Conditioning Layer*. AAAI 2018. <https://arxiv.org/abs/1709.07871>

- [4] W. Peebles, S. Xie. *Scalable Diffusion Models with Transformers (DiT)*. ICCV 2023. <https://arxiv.org/abs/2212.09748>
- [5] P. Esser et al. *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis (Stable Diffusion 3, MMDiT)*. ICML 2024. <https://arxiv.org/abs/2403.03206>
- [6] T. Park, M.-Y. Liu, T.-C. Wang, J.-Y. Zhu. *Semantic Image Synthesis with Spatially-Adaptive Normalization (SPADE)*. CVPR 2019. <https://arxiv.org/abs/1903.07291>
- [7] J. Chen et al. *PixArt- α : Fast Training of Diffusion Transformer for Photorealistic Text-to-Image Synthesis*. ICLR 2024. <https://arxiv.org/abs/2310.00426>
- [8] S. Bahmani et al. *VD3D: Taming Large Video Diffusion Transformers for 3D Camera Control*. 2024. <https://arxiv.org/abs/2407.12781>